

Hochschule Konstanz (HTWG)

Fakultät Informatik – Web-Applikationen SS26

Flappy Bird – Reimagined

Abschlussprojekt & Ausarbeitung (Meilenstein 4)

Ein Projekt von:

Alexander Engelhardt

Matrikelnummer:

313291

Studiengang:

Angewandte Informatik

Fachsemester:

5

Prüfer:

Pascal Laube

Abgabedatum:

07. August 2026

Inhaltsverzeichnis

1	Einleitung und Problemstellung	1
1.1	Historischer Kontext des Ur-Spiels	1
1.2	Projektziel: Flappy Bird – Reimagined	1
1.3	Narrativ und Storytelling	2
1.4	Zielgruppenanalyse und Anwendungsfälle	2
2	Technologie-Stack und Werkzeugkette	2
2.1	Frontend: React 19, Vite und TypeScript	2
2.2	Backend: Node.js, Express und SQLite	3
2.3	Sicherheit: Bcrypt und JWT	3
3	Systemarchitektur und Systementwurf	3
3.1	Drei-Schichten-Architektur	3
3.2	Sequenzdiagramm: Authentifizierung & Score-Übermittlung	4
3.3	Datenmodell und Relationales Schema	4
4	Frontend- und Game-Engine-Implementierung	5
4.1	Komponentenarchitektur	5
4.2	Physikalische Spielberechnung und Game Loop	5
4.2.1	Mathematische Modellierung	5
4.2.2	Hindernisgenerierung und Kollisionsdetektion	6
4.3	Theme Switching (Light- / Darkmode)	6
5	Backend-API und Sicherheitsarchitektur	6
5.1	RESTful API Spezifikation	6
5.2	JWT-Authentication Middleware	7
5.3	Sicherheitsbewertung nach OWASP Standards	7
6	Qualitätsmanagement und Testdokumentation	7
6.1	Teststrategie	7
6.2	Clientseitige Formular-Validierung	7
6.3	API-Schnittstellentests	8
7	Deployment, Containerisierung und Betrieb	8
7.1	Multi-Container-Orchestrierung via Docker	8
7.2	Schritt-für-Schritt Inbetriebnahmeanleitung	9
8	Projektfortschritt und Meilenstein-Historie	9
8.1	Meilenstein 1: Konzeption, Wireframing & Repository Setup	9
8.2	Meilenstein 2: Frontend-Entwicklung & Game-Engine	9
8.3	Meilenstein 3: Backend-REST-API & JWT-Auth	10
8.4	Meilenstein 4: Containerisierung, QA & Dokumentation	10

9 Reflexion und Ausblick	10
9.1 Lessons Learned & Erfolge	10
9.2 Herausforderungen	10
9.3 Zukünftige Erweiterungen	10
Anhang: Vollständiger Quellcode	11
A Typ-Definitionen (types.ts)	11
B Landingpage Komponente (Index.tsx)	11
C Game Engine Komponente (Game.tsx)	13
D Authentifizierungs-Seiten (Login.tsx & Register.tsx)	18
E Leaderboard Komponente (Leaderboard.tsx)	18
F Express-Backend Server (server.ts)	19
G Test-Nutzerkonten	24

Tabellenverzeichnis

1	Detaillierte Analyse des Technologie-Stacks	3
2	Spezifikation der REST-API-Endpunkte	6
3	Testmatrix der API-Integrationstests	8
4	Vordefinierte Accounts für die Evaluierung	24

Abbildungsverzeichnis

1	Schichtenarchitektur und Datenfluss der Gesamtanwendung	4
2	UML-Sequenzdiagramm des Authentifizierungs- und Score-Speicherprozesses	4
3	ERD-Darstellung der Nutzertabelle in SQLite	5

Listings

1	Theme Switch Funktionalität	6
2	JWT Verifikations-Middleware	7
3	Docker-Compose Konfiguration (compose.yaml)	8
4	Inbetriebnahme-Befehle	9
5	types.ts	11
6	Index.tsx	11
7	Game.tsx	13
8	Login.tsx	18
9	Register.tsx	18
10	Leaderboard.tsx	19
11	Auszug aus server.ts	19

1 Einleitung und Problemstellung

1.1 Historischer Kontext des Ur-Spiels

Das im Jahr 2013 vom vietnamesischen Entwickler Dong Nguyen veröffentlichte Spiel *Flappy Bird* stellte ein phänomenales Zeitdokument der mobilen Videospielgeschichte dar. Aufgrund seiner rudimentären Steuerung – ein einzelner Antipp-Impuls zur Überwindung der Gravitation – und seines extrem hohen Schwierigkeitsgrads erreichte das Spiel innerhalb kürzester Zeit hunderte Millionen Downloads.

Trotz dieses kommerziellen Phänomens wies das ursprüngliche Spiel gravierende technische und architektonische Mängel auf, die aus heutiger Sicht moderner Softwareentwicklung nicht mehr tragbar sind:

1. **Fehlende Serveranbindung:** Sämtliche Spielstände wurden ausschließlich lokal auf dem Endgerät gespeichert, was die Verifikation von Rekorden unmöglich machte.
2. **Anfälligkeit für Cheating:** Da keine serverseitige Validierung stattfand, konnten lokale Spielstände mit einfachen Speicher-Editoren manipuliert werden.
3. **Fehlendes Benutzer-Management:** Es existierte keine Möglichkeit, Benutzerkonten anzulegen, individuelle Statistiken einzusehen oder sich authentifiziert in globalen Ranglisten zu messen.
4. **Statisches visuelles Design:** Das Spiel bot keinerlei Anpassungsmöglichkeiten für Barrierefreiheit oder unterschiedliche Lichtverhältnisse (z. B. Darkmode/Lightmode).

1.2 Projektziel: Flappy Bird – Reimagined

Im Rahmen der Lehrveranstaltung *Web-Applikationen* im Sommersemester 2026 an der Hochschule Konstanz (HTWG) wurde das Projekt ****Flappy Bird – Reimagined**** ins Leben gerufen. Ziel war es, die ikonische Kernmechanik des Spiels aufzugreifen und in eine moderne, modulare, typischere Full-Stack-Webanwendung zu überführen.

Das Projekt verfolgt hierbei folgende Hauptzielsetzungen:

- **Plattformunabhängigkeit:** Vollständige Lauffähigkeit im Browser ohne zusätzliche Plugins oder native Installationen.
- **Typsicherheit über den gesamten Stack:** Durchgängiger Einsatz von TypeScript im Frontend und Backend zur Minimierung von Laufzeitfehlern.
- **Moderne Authentifizierung:** Implementierung einer zustandslosen Session-Verwaltung mittels JSON Web Tokens (JWT) und sicherer Passwort-Verschlüsselung via Bcrypt.
- **Persistente Highscore-Verwaltung:** Aufbau einer relationalen SQLite-Datenbank zur verlässlichen Speicherung von Bestenlisten.

- **Containerisiertes Deployment:** Vollständige Entkopplung der Systemkomponenten mittels Docker und Docker-Compose für ein reproduzierbares Deployment auf beliebigen Zielsystemen.

1.3 Narrativ und Storytelling

Zur Steigerung der User-Retention und Verbesserung der UI/UX-Experience wurde das Spiel um ein eigenständiges Lore-Konzept erweitert. Auf der Landingpage (`Index.tsx`) wird der Spieler in die Welt von Flappy eingeführt:

„Long ago, a small yellow bird lived in a quiet world above endless green pipes. Unlike the other birds, Flappy was never meant to stay in a cage or rest on the ground. It dreamed of the open sky and of proving that even the smallest wings could overcome impossible obstacles. One day, the bird escaped captivity and began its journey through the pipe-filled land, where every flight became a test of courage, timing, and endurance.“

1.4 Zielgruppenanalyse und Anwendungsfälle

Die Zielgruppe der Anwendung teilt sich in zwei primäre Segmente auf:

1. Competitive Casual-Gamer: Nutzer, die reaktionsbasierte, kurze Spielrunden zur Zerstreuung suchen, sich jedoch durch die globale Top-10-Bestenliste zu kontinuierlichen Verbesserungsschritten anspornen lassen. Für diese Zielgruppe stehen Performanz, geringe Latenz im Game-Loop und sofortiges Highscore-Feedback im Vordergrund.

2. Gelegenheitsnutzer & Familien: Anwender, die eine barrierefreie Webanwendung ohne Barrieren suchen. Durch den integrierten Light/Dark-Theme-Toggle wird das Spiel an unterschiedliche Umgebungsbeleuchtungen angepasst.

2 Technologie-Stack und Werkzeugkette

Der gewählte Stack basiert auf bewährten Industriestandards des JavaScript/TypeScript-Ökosystems. Tabelle 1 bietet eine komponentenweise Aufschlüsselung der eingesetzten Bibliotheken.

2.1 Frontend: React 19, Vite und TypeScript

Die Frontend-Architektur setzt auf **React 19** in Kombination mit **Vite**. Die Entscheidung für Vite anstelle klassischer Tools wie Webpack begründet sich durch das extrem schnelle Hot Module Replacement (HMR) auf Basis von nativen ES-Modulen. TypeScript fungiert als verbindendes Element über das gesamte Frontend. Durch strikte Typdefinitionen (z. B. für Pipe-Objekte oder Formular-Eigenschaften in `types.ts`) werden Refactorings und Zustandstransformationen bereits zur Kompilierzeit abgesichert.

Komponente	Technologie / Bibliothek	Version
Client-Framework	React (SPA)	19.x
Language Standard	TypeScript	5.x
Build Tool / Bundler	Vite	6.x
Client-Routing	React Router DOM	7.x
HTTP-Client	Axios	1.7.x
Backend-Server	Node.js mit Express.js	4.21.x
Passwort-Verschlüsselung	Bcrypt	5.1.x
Session-Sicherheit	JsonWebToken (JWT)	9.0.x
Relationales DBMS	SQLite (better-sqlite3)	11.x
Containerisierung	Docker & Docker-Compose	2.x

Tabelle 1: Detaillierte Analyse des Technologie-Stacks

2.2 Backend: Node.js, Express und SQLite

Als Server-Lösung kommt **Node.js** mit dem **Express.js**-Framework zum Einsatz. Die Asynchronität von Node.js prädestiniert das Framework für I/O-intensive Webapplikationen. Als Datenbank wird **SQLite** über das Node-Modul `better-sqlite3` eingebunden. Im Gegensatz zu herkömmlichen asynchronen SQLite-Treibern arbeitet `better-sqlite3` synchron, was den Codenavigationsaufwand reduziert und bei lokalen Lese-/Schreibzugriffen eine extrem hohe Durchsatzrate garantiert.

2.3 Sicherheit: Bcrypt und JWT

Zur Passwortsicherung wird der Bcrypt-Algorithmus genutzt. Durch die Inkorporierung eines variablen Salt-Faktors (standardmäßig 10 Runden) wird das System gegen Rainbow-Table-Angriffe geschützt. Das Session-Management erfolgt über JSON Web Tokens (JWT). Nach erfolgreicher Authentifizierung stellt der Server ein kryptographisch signiertes Token aus, welches vom Client bei geschützten Anfragen im `Authorization: Bearer <token>`-Header mitgeführt werden muss.

3 Systemarchitektur und Systementwurf

3.1 Drei-Schichten-Architektur

Die Webapplikation basiert auf einer entkoppelten Drei-Schichten-Architektur. Abbildung 1 visualisiert das Zusammenspiel zwischen Client-Browser, Express-Backend und SQLite-Datenbank.

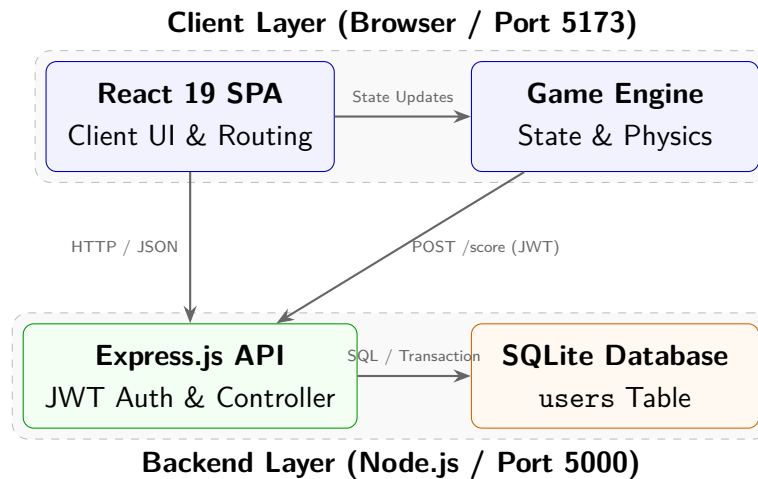


Abbildung 1: Schichtenarchitektur und Datenfluss der Gesamtanwendung

3.2 Sequenzdiagramm: Authentifizierung & Score-Übermittlung

Der Prozess der geschützten Score-Übermittlung nach einer absolvierten Spielrunde ist in Abbildung 2 als UML-Sequenzdiagramm dargestellt.

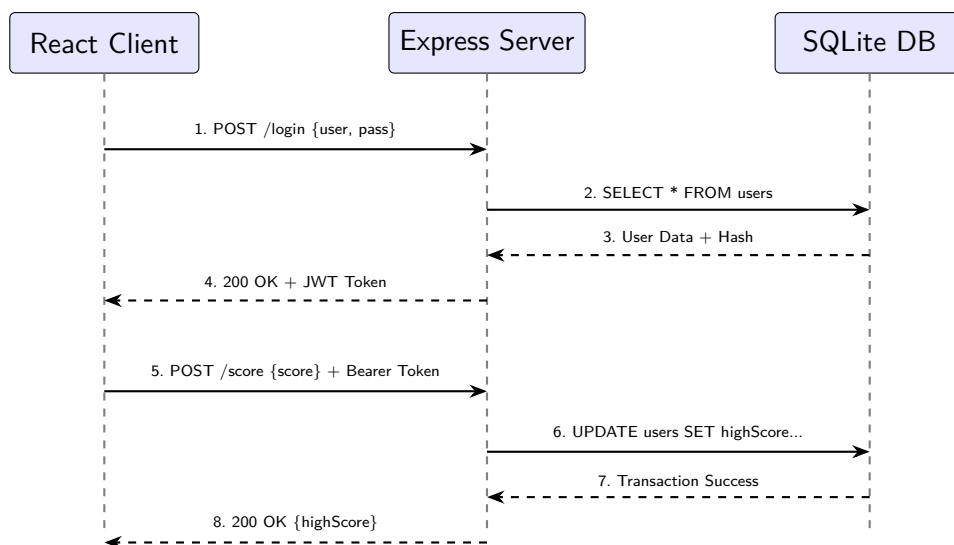


Abbildung 2: UML-Sequenzdiagramm des Authentifizierungs- und Score-Speicherprozesses

3.3 Datenmodell und Relationales Schema

Das System nutzt ein minimalistisches, aber performantes Datenbankschema. Die Tabelle users speichert alle relevanten Informationen zu Registrierungen und Rekorden.

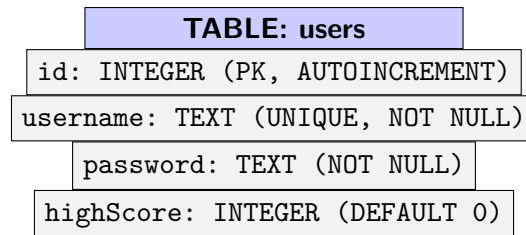


Abbildung 3: ERD-Darstellung der Nutzertabelle in SQLite

4 Frontend- und Game-Engine-Implementierung

4.1 Komponentenarchitektur

Das Frontend ist nach dem Single-Page-Application-Muster aufgebaut. Die Navigation erfolgt clientseitig ohne Neuladen der Seite über React Router v7.

- **App.tsx:** Haupt-Einstiegspunkt. Definiert die Routen (`/`, `/login`, `/register`, `/game`, `/leaderboard`) und den Zustand für den Darkmode.
- **Index.tsx:** Landingpage mit Lore-Beschreibung, Visuals und Feature-Übersicht.
- **Game.tsx:** Herzstück der Anwendung. Enthält das Render-Canvas, die physikalische Berechnungslogik und die Anbindung an die Score-API.
- **LoginForm.tsx:** Wiederverwendbare Formular-Komponente für Registrierung und Anmeldung mit integrierter Client-Validierung.
- **LeaderboardTable.tsx:** Stellt die Top 10 Spieler in einer stilisierten Tabelle dar.

4.2 Physikalische Spielberechnung und Game Loop

Die Physik des Spiels wird in der Komponente `Game.tsx` über einen zeitgesteuerten Intervall-Tick simuliert.

4.2.1 Mathematische Modellierung

Die vertikale Position des Vogels $y_t \in [0, H_{\text{game}}]$ zum diskreten Zeitschritt t errechnet sich nach der Differenzen-Gleichung:

$$y_{t+1} = y_t + v_t \quad (1)$$

Wobei die Fallgeschwindigkeit v_t kontinuierlich durch die Beschleunigung $g = 0,6$ (Gravitation) erhöht wird:

$$v_{t+1} = v_t + g \quad (2)$$

Bei einer Benutzereingabe (Leertaste oder Mausklick) wird ein konstanter negativer Sprungimpuls injiziert:

$$v_{\text{jump}} = -8,5 \quad (3)$$

4.2.2 Hindernisgenerierung und Kollisionsdetektion

Röhren-Hindernisse bewegen sich mit einer konstanten horizontalen Geschwindigkeit $v_{\text{pipe}} = 3 \text{ px/tick}$ nach links:

$$x_{\text{pipe},t+1} = x_{\text{pipe},t} - v_{\text{pipe}} \quad (4)$$

Eine Kollision wird ausgelöst, sobald der Bounding-Box-Test positiv ausfällt:

$$\text{Kollision}_{\text{Boden/Decke}} \iff y_t \leq 0 \quad \vee \quad y_t + h_{\text{bird}} \geq H_{\text{game}} \quad (5)$$

$$\begin{aligned} \text{Kollision}_{\text{Röhre}} \iff & (x_{\text{bird}} + w_{\text{bird}} > x_{\text{pipe}}) \wedge (x_{\text{bird}} < x_{\text{pipe}} + w_{\text{pipe}}) \\ & \wedge (y_{\text{bird}} < h_{\text{topPipe}} \quad \vee \quad y_{\text{bird}} + h_{\text{bird}} > h_{\text{topPipe}} + g_{\text{gap}}) \end{aligned} \quad (6)$$

Wobei $w_{\text{bird}} = 30$, $h_{\text{bird}} = 30$, $w_{\text{pipe}} = 60$, $g_{\text{gap}} = 120$ und $H_{\text{game}} = 500$ Pixel betragen.

4.3 Theme Switching (Light- / Darkmode)

Das Theme-Switching wird durch einfache DOM-Klassenmanipulation gelöst. In der Anwendung schaltet die Funktion `enableDarkmode()` die Klasse `light-mode` auf dem `body`-Element um, was wiederum globale CSS-Variablen für Hintergründe und Schriftfarben anpasst:

```
1 function enableDarkmode() {
2   document.body.classList.toggle("light-mode");
3 }
```

Listing 1: Theme Switch Funktionalität

5 Backend-API und Sicherheitsarchitektur

5.1 RESTful API Spezifikation

Das Express-Backend stellt vier zentrale Endpunkte zur Verfügung.

Endpunkt	Funktion	Auth-Level	HTTP Status
POST /api/users/register	Erstellt neues Benutzerkonto	Öffentlich	201 / 400 / 500
POST /api/users/login	Validiert Login & liefert JWT	Öffentlich	200 / 400 / 500
GET /api/game/leaderboard	Bietet Top-10 Bestenliste	Öffentlich	200 / 500
POST /api/game/score	Aktualisiert Highscore	JWT Bearer	200 / 401 / 500

Tabelle 2: Spezifikation der REST-API-Endpunkte

5.2 JWT-Authentication Middleware

Zur Absicherung geschützter Routen prüft die Middleware `authenticateToken` den eingehenden HTTP-Header auf Vorhandensein und Validität des Tokens:

```
1 const authenticateToken = (req: AuthenticatedRequest, res: Response,
2   next: NextFunction) => {
3   const authHeader = req.headers['authorization'];
4   const token = authHeader && authHeader.split(' ')[1];
5
6   if (!token) return res.status(401).json({ message: "Access Token
7     missing." });
8
9   jwt.verify(token, JWT_SECRET, (err, user) => {
10     if (err) return res.status(403).json({ message: "Invalid Token." });
11     req.user = user;
12     next();
13   });
14 }
```

Listing 2: JWT Verifikations-Middleware

5.3 Sicherheitsbewertung nach OWASP Standards

Das Sicherheitskonzept wurde gegen die gängigsten Bedrohungen abgesichert:

- **SQL Injection (SQLite):** Durch die Verwendung von Prepared Statements in `better-sqlite3` (z. B. `db.prepare(SSELECT * FROM users WHERE id = ?").get(userId)`) werden Benutzereingaben strikt vom SQL-Code getrennt. SQLite ist somit ausgeschlossen.
- **Broken Authentication:** Passwörter werden niemals im Klartext gespeichert. Der Einsatz von `Bcrypt` schützt Passwörter in der Datenbank.
- **Cross-Site Scripting (XSS):** React maskiert standardmäßig alle dynamisch gerenderten Variablen im TSX-Code, wodurch die Injektion schädlicher Skripte unterbunden wird.

6 Qualitätsmanagement und Testdokumentation

6.1 Teststrategie

Die Qualitätssicherung umfasst sowohl clientseitige Eingabevalidierungen als auch strukturierte End-to-End- und Schnittstellentests der API.

6.2 Clientseitige Formular-Validierung

In der Komponente `LoginForm.tsx` werden Eingaben vor dem Absenden eines HTTP-Requests geprüft:

- **Mindestlänge:** Passwörter müssen eine Mindestlänge von 8 Zeichen besitzen.
- **Konsistenzprüfung:** Bei der Registrierung müssen das Passwort und die Wiederholung identisch sein.
- **Checkbox-Verifikation:** Die AGB-Bestätigung ist ein zwingendes Pflichtfeld.

6.3 API-Schnittstellentests

Die API-Endpunkte wurden systematisch getestet. Tabelle 3 dokumentiert die Testergebnisse.

Szenario / Testfall		Erwarteter Status	Ergebnis	Anmerkung
Registrierung	gültiger User	201 Created	Erfolgreich	Nutzer in DB angelegt
Registrierung	existieren-der Name	400 Bad Request	Erfolgreich	Unique Constraint verhindert Duplikat
Login mit Passwort	korrektem	200 OK + Token	Erfolgreich	JWT wird im Body zurückgegeben
Login mit falschem Passwort		400 Bad Request	Erfolgreich	Zugriff verwehrt
Score senden ohne Auth-Header		401 Unauthorized	Erfolgreich	Middleware blockiert Anfrage
Score senden mit neuem Rekord		200 OK + DB Update	Erfolgreich	Highscore wird überschrieben
Score senden mit kleinerem Wert		200 OK (No Change)	Erfolgreich	Highscore bleibt unverändert

Tabelle 3: Testmatrix der API-Integrationstests

7 Deployment, Containerisierung und Betrieb

7.1 Multi-Container-Orchestrierung via Docker

Um eine identische Laufzeitumgebung für Entwicklung, Testing und Produktion zu garantieren, wurde die Anwendung containerisiert. Das Zusammenspiel von Frontend und Backend wird über `compose.yaml` gesteuert.

```

1 services:
2   backend:
3     build: .
4     container_name: flappy_backend
5     restart: always
6     ports:
7       - "5000:5000"
```

```
8   environment:
9     - PORT=5000
10    - JWT_SECRET=secret-key
11   volumes:
12     - sqlite_data:/app/data
13
14 volumes:
15   sqlite_data:
```

Listing 3: Docker-Compose Konfiguration (compose.yaml)

7.2 Schritt-für-Schritt Inbetriebnahmeanleitung

Für die Inbetriebnahme der Gesamtapplikation auf einem Zielsystem müssen lediglich folgende Befehle ausgeführt werden:

```
1 # 1. Repository klonen
2 git clone https://github.com/Guakocius/htwg.git
3 cd htwg/sem5/webapp
4
5 # 2. Multi-Container-Environment bauen und im Hintergrund ausführen
6 docker-compose up --build -d
```

Listing 4: Inbetriebnahme-Befehle

Nach dem erfolgreichen Build-Prozess stehen die Dienste wie folgt bereit:

- **Frontend Web-Applikation:** <http://localhost:5173>
- **Backend REST-API:** <http://localhost:5000>

8 Projektfortschritt und Meilenstein-Historie

Die Entwicklung erstreckte sich über vier definierte Meilensteine:

8.1 Meilenstein 1: Konzeption, Wireframing & Repository Setup

In der ersten Phase wurden die Anforderungen analysiert, Wireframes gezeichnet sowie das Mono-Repository aufgebaut. Technologische Weichenstellungen (React, Node, SQLite) wurden in dieser Phase verbindlich festgelegt.

8.2 Meilenstein 2: Frontend-Entwicklung & Game-Engine

Der Schwerpunkt des zweiten Meilensteins lag auf der Frontend-Umsetzung. Die UI-Komponenten (Landingpage, Login, Game Canvas, Leaderboard) sowie der physikalische Game-Loop in `Game.tsx` wurden vollumfänglich entwickelt.

8.3 Meilenstein 3: Backend-REST-API & JWT-Auth

Im dritten Meilenstein wurde das Express-Backend realisiert. Die Anbindung der SQLite-Datenbank, das Hashing von Passwörtern mit Bcrypt sowie die Absicherung der Highscore-Übermittlung mittels JWT standen hier im Fokus.

8.4 Meilenstein 4: Containerisierung, QA & Dokumentation

Der finale Meilenstein umfasste die Containerisierung mittels Docker-Compose, die Durchführung aller Systemtests, die Erstellung der Videodemo sowie die Ausarbeitung dieser Dokumentation.

9 Reflexion und Ausblick

9.1 Lessons Learned & Erfolge

Die durchgängige Verwendung von TypeScript im gesamten Stack hat sich als wesentlicher Erfolgsfaktor erwiesen. Typsicherheitsfehler wurden frühzeitig erkannt. Die Wahl von SQLite als eingebettete Datenbank erwies sich für das Projektvolumen als ideal, da kein komplexer Datenbank-Server administriert werden musste.

9.2 Herausforderungen

Eine Herausforderung war die Abstimmung des React-Komponenten-Lebenszyklus mit dem asynchronen Game-Loop. Da React-Zustandsupdates (useState) asynchron verarbeitet werden, mussten mutable Referenzen und funktionale State-Updates verwendet werden, um präzise Kollisionsabfragen im 24-ms-Takt zu garantieren.

9.3 Zukünftige Erweiterungen

Mögliche zukünftige Ausbaustufen der Anwendung sind:

1. **Echtzeit-Multiplayer via WebSockets:** Integration von Socket.IO, um die Positionen anderer Spieler live auf dem eigenen Spielfeld einzublenden.
2. **Server-Side Physics Processing:** Verlegung der vollständigen Physikberechnung auf den Server, um Client-seitige Schummeleien endgültig auszuschließen.
3. **In-Game Item & Skin-Shop:** Freischaltung von alternativen Charakter-Skins auf Basis der erreichten Highscores.

Anhang: Vollständiger Quellcode

A Typ-Definitionen (types.ts)

```
1 export type FormProps = {
2   form: string;
3 };
4
5 export type User = {
6   id: number;
7   name: string;
8   password: string;
9 };
```

Listing 5: types.ts

B Landingpage Komponente (Index.tsx)

```
1 import flappyImg from "../../assets/flappybird.png";
2
3 /**
4  * Default index page re-routing to the landing page.
5  */
6 export default function Index() {
7   return (
8     <>
9       <header>
10        <h1>
11          Welcome to Flappy Bird --
12          <span className="special-text">Reimagined!</span>
13        </h1>
14      </header>
15      <main id="center">
16        <div className="flappy">
17          <img
18            src={flappyImg}
19            id="flappy-img"
20            width="auto"
21            height="auto"
22            alt="a yellow pixelated bird"
23          />
24        </div>
25      </main>
26
27      <article>
28        <h2
29          style={{
30            display: "flex",
```

```

31         alignItems: "center",
32         justifyContent: "center",
33     }}
34     >
35     <u>Lore</u>
36 </h2>
37
38     <div>
39         <p>
40             Long ago, a small yellow bird lived in a quiet world above
endless
41             green pipes. Unlike the other birds, Flappy was never meant
to stay
42             in a cage or rest on the ground. It dreamed of the open sky
and of
43             proving that even the smallest wings could overcome
impossible
44             obstacles. One day, the bird escaped captivity and began its
journey
45             through the pipe-filled land, where every flight became a
test of
46             courage, timing, and endurance. With each narrow gap it
passed,
47             Flappy came closer to freedom and to becoming a legend among
birds.
48         </p>
49     </div>
50 </article>
51
52     <section>
53         <h2>Features</h2>
54         <p>
55             Flappy Bird Reimagined keeps the simple challenge of the
original game
56             while adding its own style, atmosphere, and expanded world.
57         </p>
58
59         <ul>
60             <li>Classic arcade gameplay</li>
61             <li>New visual style</li>
62             <li>Lore and worldbuilding</li>
63             <li>Leaderboard competition</li>
64         </ul>
65     </section>
66     <footer>&copy; 2026</footer>
67 </>
68 );
69 }

```

Listing 6: Index.tsx

C Game Engine Komponente (Game.tsx)

```
1 import { useState, useEffect } from "react";
2 import axios from "axios";
3 import flappyImg from "../../assets/flappybird.png";
4
5 interface Pipe {
6   x: number;
7   topHeight: number;
8   passed: boolean;
9 }
10
11 /**
12  * Page for displaying the game and handling the game's logic.
13  */
14 export default function Game() {
15   const [birdPosition, setBirdPosition] = useState<number>(250);
16   const [velocity, setVelocity] = useState<number>(0);
17   const [score, setScore] = useState<number>(0);
18   const [gameOver, setGameOver] = useState<boolean>(false);
19   const [gameStarted, setGameStarted] = useState<boolean>(false);
20   const [pipes, setPipes] = useState<Pipe[]>([]);
21
22   const gameHeight = 500;
23   const gameWidth = 400;
24   const gravity = 0.6;
25   const jumpStrength = -8.5;
26   const pipeWidth = 60;
27   const pipeGap = 120;
28   const pipeSpeed = 3;
29
30   const handleJump = () => {
31     if (!gameStarted) {
32       setGameStarted(true);
33     }
34     if (!gameOver) {
35       setVelocity(jumpStrength);
36     }
37   };
38
39   useEffect(() => {
40     const handleKeyDown = (e: KeyboardEvent) => {
41       if (e.code === "Space") {
42         handleJump();
43       }
44     };
45     window.addEventListener("keydown", handleKeyDown);
46     return () => window.removeEventListener("keydown", handleKeyDown);
47   }, [gameStarted, gameOver]);
48 }
```

```
49  useEffect(() => {
50    if (!gameStarted || gameOver) return;
51
52    const gameInterval = setInterval(() => {
53      setBirdPosition((prev) => {
54        const nextPos = prev + velocity;
55        if (nextPos >= gameHeight - 30 || nextPos <= 0) {
56          endGame();
57        }
58        return nextPos;
59      });
60      setVelocity((prev) => prev + gravity);
61
62      setPipes((prevPipes) => {
63        const updatedPipes = prevPipes
64          .map((pipe) => {
65            const nextX = pipe.x - pipeSpeed;
66
67            if (!pipe.passed && nextX + pipeWidth < 100) {
68              setScore((s) => s + 1);
69              return { ...pipe, x: nextX, passed: true };
70            }
71            return { ...pipe, x: nextX };
72          })
73          .filter((pipe) => pipe.x > -pipeWidth);
74
75      const lastPipe = updatedPipes[updatedPipes.length - 1];
76      if (!lastPipe || lastPipe.x < gameWidth - 200) {
77        const randomTopHeight =
78          Math.floor(Math.random() * (gameHeight - pipeGap - 100)) +
40;
79        updatedPipes.push({
80          x: gameWidth,
81          topHeight: randomTopHeight,
82          passed: false,
83        });
84      }
85
86      return updatedPipes;
87    });
88
89    pipes.forEach((pipe) => {
90      const birdX = 100;
91      const birdY = birdPosition;
92      const birdSize = 30;
93
94      if (birdX + birdSize > pipe.x && birdX < pipe.x + pipeWidth) {
95        if (
96          birdY < pipe.topHeight ||
97          birdY + birdSize > pipe.topHeight + pipeGap
98        ) {
```

```
99         endGame();
100     }
101 }
102 });
103 }, 24);
104
105 return () => clearInterval(gameInterval);
106 }, [gameStarted, gameOver, velocity, birdPosition, pipes]);
107
108 const endGame = () => {
109     setGameOver(true);
110     saveScore(score);
111 };
112
113 const saveScore = async (finalScore: number) => {
114     const token = sessionStorage.getItem("token");
115     if (!token) return;
116
117     try {
118         await axios.post(
119             "http://localhost:5000/api/game/score",
120             { score: finalScore },
121             { headers: { Authorization: `Bearer ${token}` } }
122         );
123     } catch (e) {
124         console.error("Failed to save high score:", e);
125     }
126 };
127
128 const restartGame = () => {
129     setBirdPosition(250);
130     setVelocity(0);
131     setScore(0);
132     setPipes([]);
133     setGameOver(false);
134     setGameStarted(true);
135 };
136
137 return (
138     <div
139         onClick={handleJump}
140         style={{
141             display: "flex",
142             flexDirection: "column",
143             alignItems: "center",
144             userSelect: "none",
145         }}
146     >
147         <header
148             id="game-points"
149             style={{ fontSize: "2rem", marginBottom: "10px" }}
150         >
```

```
150 >
151   Score: {score}
152 </header>
153 <main
154   style={{
155     position: "relative",
156     width: `${gameWidth}px`,
157     height: `${gameHeight}px`,
158     backgroundColor: "#70c5ce",
159     overflow: "hidden",
160     border: "2px solid #000",
161   }}
162 >
163   <div
164     id="player-container"
165     style={{
166       position: "absolute",
167       left: "100px",
168       top: `${birdPosition}px`,
169       width: "30px",
170       height: "30px",
171     }}
172   >
173     <img
174       src={flappyImg}
175       alt="Flappy Bird"
176       style={{ width: "100%", height: "100%" }}
177     />
178   </div>
179   <div id="pipe-container">
180     {pipes.map((pipe, index) => (
181       <div key={index}>
182         <div
183           style={{
184             position: "absolute",
185             left: `${pipe.x}px`,
186             top: 0,
187             width: `${pipeWidth}px`,
188             height: `${pipe.topHeight}px`,
189             backgroundColor: "green",
190             border: "3px solid black",
191           }}
192         />
193         <div
194           style={{
195             position: "absolute",
196             left: `${pipe.x}px`,
197             top: `${pipe.topHeight + pipeGap}px`,
198             width: `${pipeWidth}px`,
199             height: `${gameHeight - pipe.topHeight - pipeGap}px`,
200             backgroundColor: "green",
```

```
201         border: "2px solid black",
202     }}
203     />
204 </div>
205     )})
206 </div>
207
208 {!gameStarted && (
209     <div
210         style={{ textAlign: "center", marginTop: "200px", color: "#
211         fff" }}
212     >
213         <h2>Click or Press Space to Start</h2>
214     </div>
215     )}
216
217 {gameOver && (
218     <div
219         style={{
220             position: "absolute",
221             top: "30%",
222             width: "100%",
223             textAlign: "center",
224             backgroundColor: "rgba(0, 0, 0, 0.7)",
225             color: "#fff",
226             padding: "20px 0",
227         }}
228     >
229         <h2>Game Over</h2>
230         <button
231             onClick={restartGame}
232             style={{
233                 padding: "10px 20px",
234                 fontSize: "1rem",
235                 cursor: "pointer",
236             }}
237         >
238             Play Again
239         </button>
240     </div>
241     )}
242 </main>
243 </div>
244 );
}
```

Listing 7: Game.tsx

D Authentifizierungs-Seiten (Login.tsx & Register.tsx)

```
1 import LoginForm from "../components/LoginForm";
2
3 import { useNavigate } from "react-router";
4
5 /**
6  * Login page for the process of logging into an already
7  * existing account.
8  */
9 export default function Login() {
10   const navigate = useNavigate();
11
12   return (
13     <>
14       <div>
15         <button onClick={() => navigate(-1)}>go back</button>
16         <LoginForm form={"Login"} />
17       </div>
18     </>
19   );
20 }
```

Listing 8: Login.tsx

```
1 import LoginForm from "../components/LoginForm";
2
3 import { useNavigate } from "react-router";
4
5 /**
6  * Register page for the application process of a new user.
7  */
8 export default function Register() {
9   const navigate = useNavigate();
10
11   return (
12     <>
13       <div>
14         <button onClick={() => navigate(-1)}>go back</button>
15         <LoginForm form={"Register"} />
16       </div>
17     </>
18   );
19 }
```

Listing 9: Register.tsx

E Leaderboard Komponente (Leaderboard.tsx)

```
1 import { useNavigate } from "react-router";
2 import LeaderboardTable from "../components/LeaderboardTable";
3
4 /**
5  *
6  */
7 export default function Leaderboard() {
8   const navigate = useNavigate();
9
10  return (
11    <>
12      <div>
13        <button onClick={() => navigate(-1)}>go back</button>
14        <LeaderboardTable />
15      </div>
16    </>
17  );
18 }
```

Listing 10: Leaderboard.tsx

F Express-Backend Server (server.ts)

```
1 import express from "express";
2 import type { Request, Response, NextFunction } from "express";
3 import cors from "cors";
4 import bcrypt from "bcrypt";
5 import jwt from "jsonwebtoken";
6 import dotenv from "dotenv";
7 import Database from "better-sqlite3";
8 import fs from "fs";
9 import { fileURLToPath } from "url";
10 import path from "path";
11
12 dotenv.config();
13
14 const app = express();
15 const PORT = process.env.PORT || 5000;
16 const JWT_SECRET = process.env.JWT_SECRET || "secret-key";
17
18 app.use(
19   cors({
20     origin: ["http://localhost:5173", "http://127.0.0.1:5173"],
21     credentials: true,
22     methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
23     allowedHeaders: ["Content-Type", "Authorization"],
24   }),
25 );
```

```
26 app.use(express.json());
27
28 const __filename = fileURLToPath(import.meta.url);
29 const __dirname = path.dirname(__filename);
30
31 const dbDir = path.join(__dirname, "../../data");
32 if (!fs.existsSync(dbDir)) {
33   fs.mkdirSync(dbDir, { recursive: true });
34 }
35
36 const db = new Database(path.join(dbDir, "flappybird.db"));
37
38 db.exec(`
39     CREATE TABLE IF NOT EXISTS users (
40         id INTEGER PRIMARY KEY AUTOINCREMENT,
41         username TEXT UNIQUE NOT NULL,
42         password TEXT NOT NULL,
43         highScore INTEGER DEFAULT 0
44     )
45 `);
46
47 console.log("Connected to SQLite successfully.");
48
49 interface UserPayload {
50   id: number;
51   username: string;
52 }
53
54 interface AuthenticatedRequest extends Request {
55   user?: UserPayload | jwt.JwtPayload;
56 }
57
58 const authenticateToken = (
59   req: AuthenticatedRequest,
60   res: Response,
61   next: NextFunction,
62 ): void => {
63   const authHeader = req.headers["authorization"];
64   const token = authHeader && authHeader.split(" ")[1];
65
66   if (!token) {
67     res.status(401).json({ message: "Access denied. No token provided." });
68     return;
69   }
70
71   try {
72     const verified = jwt.verify(token, JWT_SECRET) as UserPayload;
73     req.user = verified;
74     next();
75   } catch (e) {
```



```
76     res.status(403).json({ message: 'Invalid or expired token: ${e}' });
77   }
78 };
79
80 app.post("/api/users/register", async (req: Request, res: Response) => {
81   try {
82     const { username, password } = req.body;
83
84     if (!username || !password) {
85       return res
86         .status(400)
87         .json({ message: "Username and password are required." });
88     }
89
90     const userExists = db
91       .prepare("SELECT * FROM users WHERE username = ?")
92       .get(username);
93
94     if (userExists) {
95       return res.status(400).json({ message: "Username is already taken."
96     });
97     }
98
99     const salt = await bcrypt.genSalt(10);
100     const hashedPassword = await bcrypt.hash(password, salt);
101
102     const stmt = db.prepare(
103       "INSERT INTO users (username, password) VALUES (?, ?)",
104     );
105     stmt.run(String(username), String(hashedPassword));
106
107     return res.status(201).json({ message: "User registration successful."
108   });
109   } catch (e) {
110     console.error("Registration error:", e);
111     return res.status(500).json({ message: "Server error saving user
112   data." });
113   }
114 });
115
116 app.post("/api/users/login", async (req: Request, res: Response) => {
117   try {
118     const { username, password } = req.body;
119
120     const user = db
121       .prepare("SELECT * FROM users WHERE username = ?")
122       .get(username) as
```

```
    | { id: number; username: string; password: string; highScore:
    number }
    | undefined;
```

```
123     if (!user) {
124         return res.status(400).json({ message: "Invalid username or
password." });
125     }
126
127     const validPassword = bcrypt.compareSync(password, user.password);
128     if (!validPassword) {
129         return res.status(400).json({ message: "Invalid username or
password" });
130     }
131
132     const token = jwt.sign(
133         { id: user.id, username: user.username },
134         JWT_SECRET,
135         { expiresIn: "1h" },
136     );
137
138     return res.status(200).json({
139         message: "Login successful.",
140         token,
141         user: {
142             id: user.id,
143             username: user.username,
144             highScore: user.highScore,
145         },
146     });
147 } catch (e) {
148     console.error("Login error:", e);
149     res.status(500).json({ message: "Server error during login." });
150 }
151 });
152
153 app.get("/api/game/leaderboard", async (_req: Request, res: Response) =>
{
154     try {
155         const leaderboard = db
156             .prepare(
157                 "SELECT id as _id, username, highScore FROM users ORDER BY
highScore DESC LIMIT 10",
158             )
159             .all();
160
161         return res.status(200).json(leaderboard);
162     } catch (e) {
163         console.error("Leaderboard error:", e);
164         res.status(500).json({ message: "Server error fetching leaderboard."
});
165     }
166 });
167
168 app.post(
```

```
169   "/api/game/score",
170   authenticateToken,
171   async (req: AuthenticatedRequest, res: Response) => {
172     try {
173       const { score } = req.body;
174       const userId = req.user?.id;
175
176       if (!userId) {
177         res.status(401).json({ message: "Unauthorized." });
178         return;
179       }
180
181       const user = db
182         .prepare("SELECT * FROM users WHERE id = ?")
183         .get(userId) as
184         | { id: number; username: string; highScore: number }
185         | undefined;
186
187       if (!user) {
188         res.status(404).json({ message: "User not found." });
189         return;
190       }
191
192       if (score > user.highScore) {
193         db.prepare("UPDATE users SET highScore = ? WHERE id = ?").run(
194           score,
195           userId,
196         );
197         return res
198           .status(200)
199           .json({ message: "New high score!", highScore: user.highScore
200             });
201       }
202
203       return res
204         .status(200)
205         .json({ message: "Score processed.", highScore: user.highScore
206           });
207     } catch (e) {
208       console.error("Score error:", e);
209       res.status(500).json({ message: "Server error updating score." });
210     }
211   },
212 );
213
214 app.listen(PORT, () => {
215   console.log('Backend server running on port ${PORT}');
216 });
```

Listing 11: Auszug aus server.ts

G Test-Nutzerkonten

Tabelle 4: Vordefinierte Accounts für die Evaluierung

Rolle	Benutzername	Passwort
Standard-User	testuser	Password123!
Leaderboard Top-Player	player1	SecurePass123